

COMP3308/3608 Week 7 - Decision Trees

Author
K. Yu
2026

Contents

1	Decision Trees	3
1.1	Basic Idea	3
1.2	Decision Trees as Rules	3
2	Constructing Decision Trees	4
2.1	ID3 Algorithm	4
2.2	Stopping Criterion	4
3	Choosing the best attribute	5
3.1	Purity	5
3.2	Entropy	5
4	Information Gain	6
4.1	Definition	6
4.2	Intuition	6
5	Decision Tree Learning as Search	7
5.1	Hypothesis Space	7
5.2	Greedy Search	7
6	Expressiveness of Decision Trees	7
6.1	Decision Trees can express any function	7
6.2	Compact trees are preferred	7
7	Decision Boundary of Decision Trees	8
7.1	Axis-aligned splits	8
7.2	Piecewise rectangular regions	8
8	Overfitting in Decision Trees	8
8.1	Definition of overfitting	8
8.2	Why Decision Trees overfit	9

9 Pruning	9
9.1 Motivation	9
9.2 Pre-Pruning	10
9.3 Post-Pruning	10
10 Validation Set	11
10.1 Data split	11
10.2 Why validation helps?	11
11 Numeric Attributes	11
11.1 Discretisation	11
11.2 Candidate split points	11
12 Bias of Information Gain	12
12.1 Problem with highly branching attributes	12
12.2 Gain Ratio	13
13 Missing Values	13
13.1 Missing values during training	13
13.2 Missing values during classification	14
14 Attributes with different Costs	14
14.1 Motivation	14
14.2 Cost-sensitive gain	14
15 Computational Cost	14
15.1 Building and Pruning Cost	14

1 Decision Trees

1.1 Basic Idea

A decision tree is a supervised learning method in a tree structure.

- Each **internal node** tests one attribute.
- Each **branch** corresponds to one possible value or condition of that attribute.
- Each **leaf node** assigns a class label.

For a new example $\mathbf{x} \in \mathbb{R}^d$, prediction is made by starting at the root, following the branch whose condition matches $\mathbf{x}$, and returning the class label at the leaf.

Mathematically, a decision tree represents a function

$$h : \mathbb{R}^d \rightarrow \mathcal{Y},$$

where $\mathcal{Y}$ is the set of possible class labels.

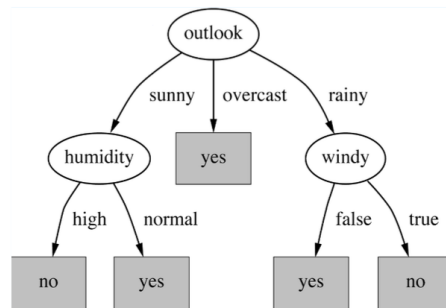


Figure 1: Weather data decision tree classifier.

1.2 Decision Trees as Rules

A decision tree can also be written as a set of rules. Each root-to-leaf path corresponds to one rule. Each rule is a conjunction of tests. For example shown in Figure 1, if outlook = sunny and humidity = high, then play = no.

The whole tree is a disjunction of these path rules:

$$\text{DT} = \text{rule}_1 \vee \text{rule}_2 \vee \cdots \vee \text{rule}_m.$$

So a decision tree is essentially a disjunction of conjunctions.

2 Constructing Decision Trees

2.1 ID3 Algorithm

Given the training set $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, the goal is to build a decision tree that correctly classifies the training examples and hopefully **generalises** to unseen examples.

ID3 builds the tree **top-down** using recursive divide-and-conquer.

At each node:

1. Choose the **best attribute** to split the examples.
2. Create **one branch for each** possible value of the attribute.
3. Split the examples into **subsets** according to the branch they follow.
4. Repeat recursively for each subset.
5. Stop when a **stopping criterion** is satisfied.

Mathematically, at a node containing a subset $S \subseteq \{1, \dots, n\}$, we choose an attribute A and partition S into

$$S_v = \{i \in S : A(\mathbf{x}^{(i)}) = v\} \quad \text{for each value } v \in \text{Values}(A).$$

The recursive process continues on each S_v .

2.2 Stopping Criterion

The simplest stopping case is $y^{(i)} = y^{(j)}$ for all $i, j \in S$. i.e. all examples reaching the current node have the same class. Then we create a leaf node with that class.

More generally, stopping happens in three cases.

1. All examples in the current subset have the **same class**. Then return that class.
2. Examples **cannot be split further** because all remaining examples have the same attribute values but different classes. This usually indicates noise. Then return the majority class of the current subset.
3. The current subset is **empty**. Then return the majority class of the parent subset.

The majority class rule is

$$\text{majority}(S) = \operatorname{argmax}_{c \in \mathcal{Y}} \left| \left\{ i \in S : y^{(i)} = c \right\} \right|.$$

3 Choosing the best attribute

3.1 Purity

At each node, we want to choose the attribute that makes the child nodes as **pure** as possible.

Definition 3.1.1 (Purity). A node is **pure** if most or all of its examples belong to the same class. For a subset S , if all examples have the same class, then S is **completely pure**. If the classes are evenly mixed, then S is **impure**.

Decision tree learning needs a quantitative measure of impurity. Entropy comes in mind.

3.2 Entropy

Definition 3.2.1 (Entropy). Let S be a subset of training examples. Suppose there are K possible classes, $\mathcal{Y} = \{c_1, c_2, \dots, c_K\}$. Define

$$p_k = \frac{|\{i \in S : y^{(i)} = c_k\}|}{|S|}.$$

The **entropy** of S is

$$H(S) = - \sum_{k=1}^K p_k \log p_k.$$

By convention, $p_k \log_2 p_k = 0$ if $p_k = 0$.

Entropy measures **class disorder**. If S is pure, then one class has probability 1, all others have probability 0, so $H(S) = 0$. If the classes are evenly distributed, entropy is high.

For binary classification with positive proportion p , negative proportion $1 - p$,

$$H(S) = -p \log_2 p - (1 - p) \log_2 (1 - p).$$

So for binary classification, $H(S) \in [0, 1)$. The maximum is achieved at $p = 1/2$. That is, entropy is highest when the node is evenly split between the two classes.

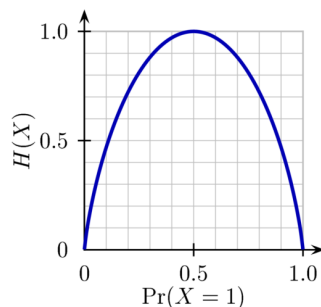


Figure 2: Binary entropy function.

4 Information Gain

Information gain measures how much entropy is **reduced by splitting** on an attribute.

4.1 Definition

Definition 4.1.1 (Information gain). Let A be an attribute. Splitting S by A creates subsets $S_v = \{i \in S : A(\mathbf{x}^{(i)}) = v\}$, $v \in \text{Values}(A)$. The entropy after splitting by A is the weighted average of child entropies:

$$H(S|A) = \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v).$$

Then **information gain** is

$$\text{Gain}(S|A) = H(S) - H(S|A).$$

Equivalently,

$$\text{Gain}(S|A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v).$$

The best attribute is chosen by

$$A^* = \underset{A}{\operatorname{argmax}} \text{Gain}(S|A).$$

So ID3 greedily chooses the attribute that gives the **greatest reduction in entropy**.

4.2 Intuition

The term $H(S)$ is the impurity before splitting. The term $H(S|A)$ is the expected impurity after splitting by A . Therefore, $\text{Gain}(S|A)$ is the improvement obtained by using A .

If an attribute creates pure child nodes, then the child entropies are small, so $H(S|A)$ is small and $\text{Gain}(S|A)$ is large.

If an attribute does not separate the classes well, then the child nodes remain impure, so $H(S|A)$ is close to $H(S)$, and the gain is small.

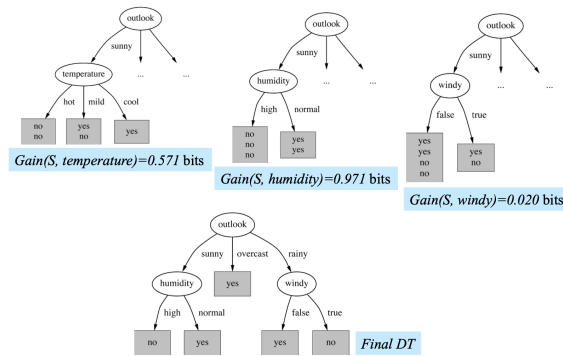


Figure 3: Split of weather data decision tree based on information gain.

5 Decision Tree Learning as Search

5.1 Hypothesis Space

A decision tree is a hypothesis $h \in H$, where H is the space of possible decision trees. The learning algorithm searches for a tree h that fits the training data:

$$h(\mathbf{x}^{(i)}) = y^{(i)}$$

for as many training examples as possible.

In the ideal noise-free case, the goal is to find

$$h(\mathbf{x}^{(i)}) = y^{(i)} \quad \forall i = 1, \dots, n.$$

5.2 Greedy Search

ID3 performs greedy search. At each node, it chooses the locally best attribute:

$$A^* = \operatorname{argmax}_A \operatorname{Gain}(S | A).$$

This is a hill-climbing style search where information gain is the evaluation function. The algorithm does not backtrack. Once an attribute is chosen at a node, the algorithm recursively continues below that node.

Therefore, ID3 is efficient, but it is not guaranteed to find the globally optimal decision tree.

6 Expressiveness of Decision Trees

6.1 Decision Trees can express any function

Decision trees are highly expressive. For any finite training set, there exists a decision tree that perfectly classifies the training examples, unless the target function is nondeterministic.

A simple construction is a lookup-table tree: create one root-to-leaf path for each training example.

This means decision trees can represent very complex functions.

6.2 Compact trees are preferred

Although a large lookup-table tree can fit the training data, it may generalise poorly. A compact tree is preferred because it is more likely to capture meaningful patterns rather than memorise individual examples.

This is an important bias of decision tree learning: *prefer small trees that explain the data well.*

7 Decision Boundary of Decision Trees

7.1 Axis-aligned splits

For numeric attributes, decision trees usually create threshold tests such as $x_j < t$ or $x_j \geq t$. Each test splits the feature space into axis-aligned regions.

For example, if

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix},$$

then a test $x_1 < t$ creates a **vertical** decision boundary, while $x_2 < t$ creates a **horizontal** decision boundary.

Therefore, decision trees create piecewise axis-aligned decision regions.

7.2 Piecewise rectangular regions

A decision tree recursively partitions the input space. Each leaf corresponds to a region of the feature space. In two dimensions, these regions are usually rectangles or unions of rectangles.

Compared with a linear classifier, a decision tree can create nonlinear decision boundaries by combining multiple axis-aligned splits.

However, the boundary is still constrained by the structure of the tree.

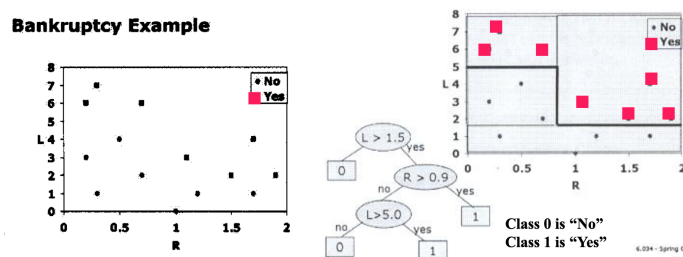


Figure 4: Decision boundary of a binary classification problem with numeric attributes. R is the ratio of earnings to expenses; L denotes the number of late payments on credit cards over the past year.

8 Overfitting in Decision Trees

8.1 Definition of overfitting

Overfitting occurs when a model fits the training data well but performs poorly on unseen data.

Definition 8.1.1 (Overfitting). Given hypothesis space H , training data $\mathcal{D}_{\text{train}}$, and full data distribution $\mathcal{D}$, a hypothesis $h \in H$ **overfits** the training data if there exists another hypothesis $h' \in H$ such that

$$\text{Error}_{\text{train}}(h) < \text{Error}_{\text{train}}(h')$$

but

$$\text{Error}(h) > \text{Error}(h').$$

So h looks better on the training set but worse on the true distribution.

8.2 Why Decision Trees overfit

Decision trees can grow deep enough to perfectly classify the training data. If the training set is small or noisy, this can produce a tree that captures accidental patterns rather than true structure. Reasons include:

- Training data may be too small.
- Training labels may contain noise.
- The tree may keep splitting until each leaf is too specific.
- The tree may behave like a lookup table.

In this case, the tree has memorised the training data rather than learned the underlying pattern.

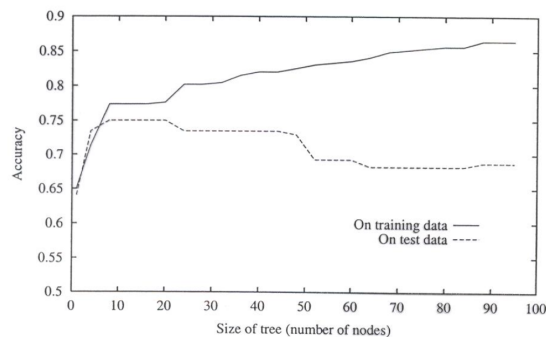


Figure 5: Large trees may overfit.

9 Pruning

9.1 Motivation

Pruning is used to reduce overfitting. The idea is to remove parts of the tree that fit noise or accidental patterns in the training set. There are two main approaches:

1. **Pre-pruning** stops tree growth early.
2. **Post-pruning** grows the full tree first, then removes unnecessary subtrees.

Post-pruning is generally more successful and widely used.

9.2 Pre-Pruning

Pre-pruning stops splitting before the tree perfectly classifies the training set. For example we may stop if $\text{Gain}(S|A)$ is too small, or the number of examples in a node is below some threshold. The risk is stopping too early, which may cause underfitting.

9.3 Post-Pruning

Post-pruning first grows the full tree, then simplifies it. A common method is subtree replacement.

For a candidate internal node:

1. Remove the subtree rooted at the node.
2. Replace it with a leaf.
3. Label the leaf with the majority class of examples reaching that node.
4. Compare the pruned tree with the original tree using a validation set.
5. Keep the pruning if validation accuracy improves or stays the same.

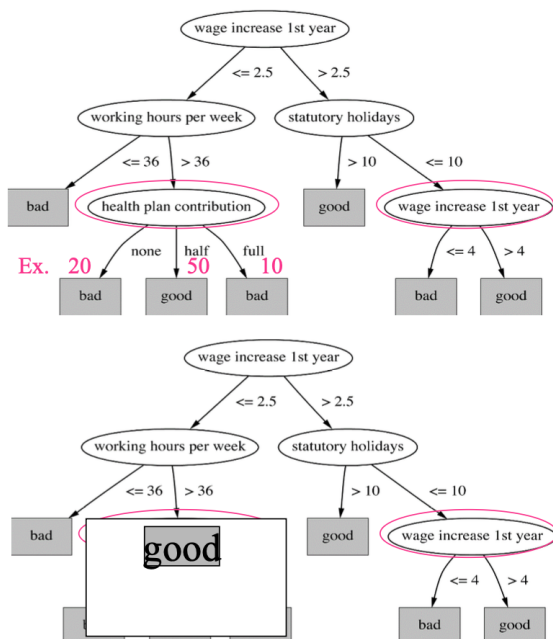


Figure 6: An example of post-pruning.

Mathematically, if node N receives subset S_N , replace its subtree with

$$\text{majority}(S_N) = \underset{c \in \mathcal{Y}}{\operatorname{argmax}} \left| \left\{ i \in S_N : y^{(i)} = c \right\} \right|.$$

The pruning is accepted if

$$\text{Acc}_{\text{val}}(T_{\text{pruned}}) \geq \text{Acc}_{\text{val}}(T_{\text{original}}).$$

10 Validation Set

10.1 Data split

To decide how much pruning to do, the available data can be split into three parts:

- training set;
- validation set;
- test set.

Training set is used to build the tree. Validation set is used to decide pruning. Test set is used only at the end to estimate final performance.

A typical split is 50% training, 25% validation, and 25% test.

10.2 Why validation helps?

The training set may contain random noise or accidental regularities. A complex tree may fit these patterns, but they are unlikely to repeat in the validation set.

Therefore, validation performance gives a safer estimate of generalisation than training performance.

However, if data is limited, setting aside validation data reduces the number of examples available for training.

11 Numeric Attributes

11.1 Discretisation

Decision trees can handle numeric attributes by converting them into threshold-based binary tests. For a numeric attribute A , we consider splits of the form $A < t$ and $A \geq t$. The threshold t is chosen by information gain or another splitting criterion.

11.2 Candidate split points

The procedure is:

1. Sort the training examples by the numeric attribute.
2. Identify adjacent examples with different class labels.
3. Generate candidate split points halfway between the adjacent values.
4. Evaluate information gain for each candidate threshold.

5. Choose the threshold with the highest gain.
6. Suppose the sorted values contain two adjacent examples a_r, a_{r+1} with different class labels. Then a candidate threshold is

$$t = \frac{a_r + a_{r+1}}{2}.$$

7. For each candidate t , split

$$S_{<t} = \left\{ i \in S : A(\mathbf{x}^{(i)}) < t \right\},$$

$$S_{\geq t} = \left\{ i \in S : A(\mathbf{x}^{(i)}) \geq t \right\}.$$

Then compute

$$\text{Gain}(S \mid A < t) = H(S) - \frac{|S_{<t}|}{|S|} H(S_{<t}) - \frac{|S_{\geq t}|}{|S|} H(S_{\geq t}).$$

8. Choose the t with largest gain:

$$t^* = \underset{t}{\operatorname{argmax}} \text{Gain}(S \mid A < t).$$

12 Bias of Information Gain

12.1 Problem with highly branching attributes

Information gain is biased toward attributes with many possible values.

For example, an ID attribute may have a unique value for every example. If we split by ID, every child node contains only one example, so every child node is pure. Therefore,

$$H(S \mid \text{ID}) = 0.$$

Then

$$\text{Gain}(S \mid \text{ID}) = H(S).$$

This looks very good according to information gain, but it is useless for generalisation. The tree simply memorises the training examples.

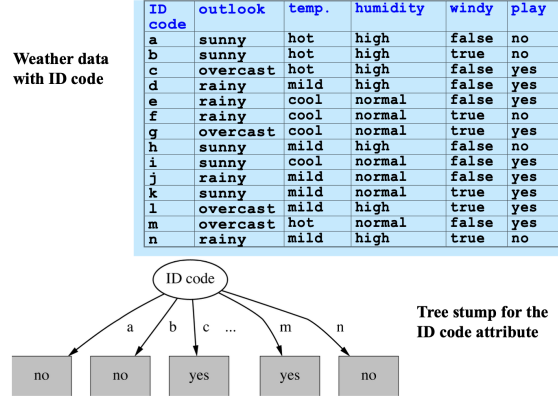


Figure 7: Example of highly-branching attributes.

12.2 Gain Ratio

Gain ratio modifies information gain to reduce the bias toward highly branching attributes.

First define:

$$\text{SplitInformation}(S | A) = - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}.$$

Then gain ratio is

$$\text{GainRatio}(S | A) = \frac{\text{Gain}(S | A)}{\text{SplitInformation}(S | A)}$$

The denominator penalises attributes that split the data into many small branches. This makes highly branching attributes less attractive.

13 Missing Values

13.1 Missing values during training

When building a decision tree, some examples may have missing values for the attribute being tested.

Suppose we are at node N , and we want to split by attribute A . Some examples reaching N have missing values for A . C4.5-style handling distributes the example fractionally according to the observed proportions among non-missing examples.

For example, if among examples with known A ,

$$P(A = v_1) = 0.6, \quad P(A = v_2) = 0.4,$$

then an example with missing A is split as 0.6 of the example down branch v_1 , and 0.4 of the example down branch v_2 . These fractional examples are then used in entropy and information gain calculations.

13.2 Missing values during classification

The same idea can be used when classifying a new example with missing values. If the test at a node uses an attribute whose value is missing, the example can be sent fractionally down multiple branches according to branch probabilities. The final prediction can be combined from the leaves reached.

14 Attributes with different Costs

14.1 Motivation

In some tasks, attributes have different costs. For example, in medical diagnosis, `temperature`, `biopsyResult`, `pulse`, `bloodTestResult` may have very different monetary, time, or patient-comfort costs.

A useful tree should prefer low-cost attributes when possible and only use expensive attributes when needed.

14.2 Cost-sensitive gain

A method to penalise high-cost attributes is to modify the gain. The general idea is prefer attributes with **high gain** and **low cost**. So instead of selecting

$$A^* = \operatorname{argmax}_A \operatorname{Gain}(S | A),$$

we select an attribute using a cost-adjusted score, such as

$$\frac{\operatorname{Gain}(S | A)}{\operatorname{Cost}(A)}$$

or other more refined penalties. The goal is to trade off predictive usefulness and measurement cost.

15 Computational Cost

15.1 Building and Pruning Cost

Let n be the number of training examples, and m be the number of attributes. The cost of building a decision tree is $O(mn \log n)$.

For pruning, subtree replacement pruning has cost $O(n)$. Subtree lifting has cost $O(n(\log n)^2)$.

So the total cost is approximately

$$O(mn \log n) + O(n(\log n)^2).$$